1ÈRE NSI · FICHE RÉSUMÉ

La logique *booléenne*

1 Les booléens

- Un **booléen** ne peut prendre que **deux valeurs** : `True` (vrai, 1) ou `False` (faux, 0). Son type est `bool`.
- Les **comparaisons** `==` `!=` `<` `<=` `>` `>=` produisent des booléens.
- Une **condition** de `if` ou de `while` est une **expression booléenne**.
- `True` s'écrit avec une majuscule et **sans guillemets** : `"True"` est une chaîne.

```
majeur = age >= 18   # True ou False
```

2 Les opérateurs not, and, or

- `not a` : vrai si a est **faux** (il inverse la valeur).
- `a and b` : vrai si a et b sont vrais **tous les deux**.
- `a or b` : vrai si **au moins un** des deux est vrai (ou **inclusif**).

a	b	not a	a and b	a or b
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

NOTATIONS

FRANÇAIS	PYTHON	LOGIQUE	ÉLECTRONIQUE
NON a	`not a`	¬a	$\bar{a}$
a ET b	`a and b`	a ∧ b	a · b
a OU b	`a or b`	a ∨ b	a + b

PRIORITÉS

not, puis **and**, puis **or** (comme × avant +). Dans le doute, des parenthèses.

```
True or False and False      # → True
```

3 Le ou exclusif (xor)

- **a xor b** (a ⊕ b) : vrai si **exactement un** des deux est vrai.
- En Python, avec deux booléens : `a != b`.

a	b	a xor b
0	0	0
0	1	1
1	0	1
1	1	0

a + b	retenue	somme
0 + 0	0	0
0 + 1	0	1
1 + 0	0	1
1 + 1	1	0

> Addition de deux bits : **somme = a xor b** et **retenue = a and b** (demi-additionneur).

4 Table de vérité d'une expression

- Avec **n variables** : **2^n lignes** (4 pour 2 variables, 8 pour 3, 16 pour 4).
- Combinaisons dans l'ordre du binaire : 000, 001, 010, … , 111.
- Une colonne par **sous-expression**, en respectant les priorités ; la dernière donne le résultat.

EXEMPLE : E = (A AND NOT B) OR C

a	b	c	not b	a and not b	E
0	0	0	1	0	**0**
0	0	1	1	0	**1**
0	1	0	0	0	**0**
0	1	1	0	0	**1**
1	0	0	1	1	**1**
1	0	1	1	1	**1**
1	1	0	0	0	**0**
1	1	1	0	0	**1**

> Deux expressions sont **équivalentes** si elles ont **la même table de vérité**.

FAIRE DRESSER LA TABLE PAR PYTHON

```
for a in [False, True]:
    for b in [False, True]:
        print(a, b, a and b, a or b)
```

5 Les lois de De Morgan

`not (a and b)` ⟺ `not a or not b`
`not (a or b)` ⟺ `not a and not b`

La négation d'un **ET** est le **OU** des négations, et inversement.

a	b	not (a and b)	not a or not b
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

ÉCRIRE LE CONTRAIRE D'UNE CONDITION

CONDITION	SON CONTRAIRE
`x < 10`	`x >= 10`
`x == 0`	`x != 0`
`n >= 10 and n <= 20`	`n < 10 or n > 20`
`eau < 2 or jours > 5`	`eau >= 2 and jours <= 5`

Le contraire de `<` est `>=`, pas `>`.

LE PIÈGE DU « NI… NI… »

```
# ✓ correct
if jour != "samedi" and jour != "dimanche":
# ✗ toujours vrai
if jour != "samedi" or jour != "dimanche":
```

CONDITION D'UN WHILE

On écrit le **contraire de la condition d'arrêt**. On s'arrête si le code est bon **ou** s'il n'y a plus d'essai, donc :

```
while code != "1234" and essais > 0:
```

AUTRES PROPRIÉTÉS

- `not (not a)` vaut `a` ; `a and b` vaut `b and a`.
- `a and True` vaut `a` ; `a or True` vaut `True` ; `a and False` vaut `False`.

Je sais…

- ☐ reconnaître une valeur et une expression booléennes
- ☐ donner la table de `not`, `and`, `or`, xor
- ☐ évaluer une expression avec les priorités
- ☐ dresser la table d'une expression à 2 ou 3 variables
- ☐ reconnaître deux expressions équivalentes
- ☐ écrire le contraire d'une condition (De Morgan)
- ☐ expliquer l'évaluation paresseuse
- ☐ écrire une fonction `est_…` qui renvoie une condition

6 L'évaluation paresseuse

`and` et `or` sont évalués **de gauche à droite** et **s'arrêtent dès que le résultat est connu** :

- `False and …` vaut `False` : la droite n'est pas évaluée.
- `True or …` vaut `True` : la droite n'est pas évaluée.

```
x = 0
x != 0 and 10 / x > 1   # False
10 / x > 1 and x != 0   # erreur !
```

On place d'abord la condition **de sécurité** : celle qui garantit que la suite peut être calculée.

7 Les fonctions booléennes

- Elles renvoient `True` ou `False` ; leur nom commence par **est**.
- La condition est déjà un booléen : on la **renvoie directement**.

```
def est_pair(n):
    return n % 2 == 0

def est_bissextile(annee):
    div4 = annee % 4 == 0
    div100 = annee % 100 == 0
    div400 = annee % 400 == 0
    return (div4 and not div100) or div400
```

- On écrit `if est_pair(n):`, et non `if est_pair(n) == True:`.
- On teste les **cas limites** : `est_bissextile(1900)` → `False`, `est_bissextile(2000)` → `True`.

! Les pièges classiques

- Le **OU logique est inclusif** : `True or True` vaut `True`.
- **and** passe avant **or** : `a or b and c` se lit `a or (b and c)`.
- Contraire de `<` : `>=`. Contraire de `==` : `!=`.
- « ni A ni B » : `not A and not B`, jamais `or`.
- Pour nier un `and`, il faut un `or` (et inversement).
- L'**ordre** des conditions compte (évaluation paresseuse).
- `"True"` (chaîne) n'est pas `True` (booléen).
- Inutile d'écrire `== True` ou `if … : return True else: return False`.